cost 1, but a shorter path exists by following the loop v_5, v_4, v_2, v_5, v_4, which has cost -5. This path is still not the shortest, because we could stay in the loop arbitrarily long. Thus, the shortest path between these two points is undefined. Similarly, the shortest path from v_1 to v_6 is undefined, because we can get into the same loop. This loop is known as a **negative-cost cycle;** when one is present in the graph, the shortest paths are not defined. Negative-cost edges are not necessarily bad, as the cycles are, but their presence seems to make the problem harder. For convenience, in the absence of a negative-cost cycle, the shortest path from s to s is zero.

There are many examples where we might want to solve the shortest-path problem. If the vertices represent computers; the edges represent a link between computers; and the costs represent communication costs (phone bill per a megabyte of data), delay costs (number of seconds required to transmit a megabyte), or a combination of these and other factors, then we can use the shortest-path algorithm to find the cheapest way to send electronic news from one computer to a set of other computers.

We can model airplane or other mass transit routes by graphs and use a shortest-path algorithm to compute the best route between two points. In this and many practical applications, we might want to find the shortest path from one vertex, s, to only one other vertex, t. Currently there are no algorithms in which finding the path from s to one vertex is any faster (by more than a constant factor) than finding the path from s to all vertices.

We will examine algorithms to solve four versions of this problem. First, we will consider the unweighted shortest-path problem and show how to solve it in $O(|E|+|V|)$. Next, we will show how to solve the weighted shortest-path problem if we assume that there are no negative edges. The running time for this algorithm is $O(|E| \log |V|)$ when implemented with reasonable data structures.

If the graph has negative edges, we will provide a simple solution, which unfortunately has a poor time bound of $O(|E| \cdot |V|)$. Finally, we will solve the weighted problem for the special case of acyclic graphs in linear time.

9.3.1 Unweighted Shortest Paths

Figure 9.10 shows an unweighted graph, G. Using some vertex, s, which is an input parameter, we would like to find the shortest path from s to all other vertices. We are only interested in the number of edges contained on the path, so there are no weights on the edges. This is clearly a special case of the weighted shortest-path problem, since we could assign all edges a weight of 1.

For now, suppose we are interested only in the length of the shortest paths, not in the actual paths themselves. Keeping track of the actual paths will turn out to be a matter of simple bookkeeping.

Suppose we choose s to be v_3. Immediately, we can tell that the shortest path from s to v_3 is then a path of length 0. We can mark this information, obtaining the graph in Figure 9.11.

Now we can start looking for all vertices that are a distance 1 away from s. These can be found by looking at the vertices that are adjacent to s. If we do this, we see that v_1 and v_6 are one edge from s. This is shown in Figure 9.12.

We can now find vertices whose shortest path from s is exactly 2, by finding all the vertices adjacent to v_1 and v_6 (the vertices at distance 1), whose shortest paths are not

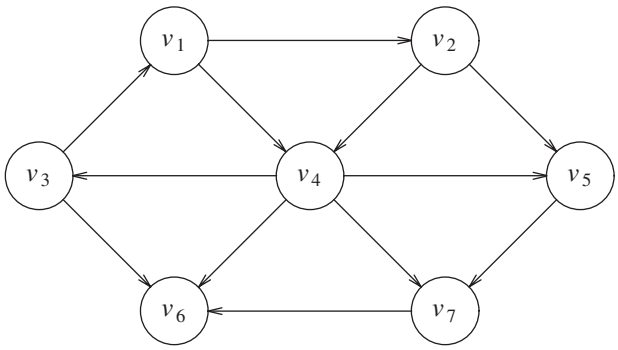

Figure 9.10 An unweighted directed graph G

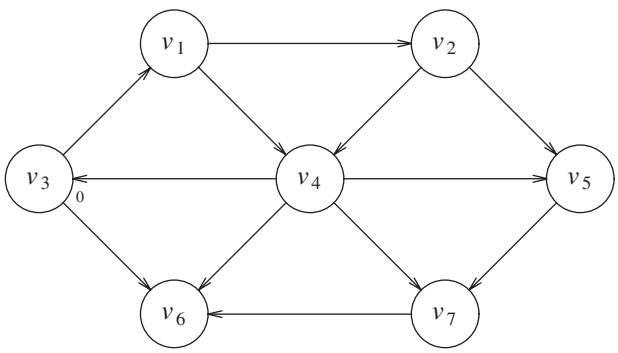

Figure 9.11 Graph after marking the start node as reachable in zero edges

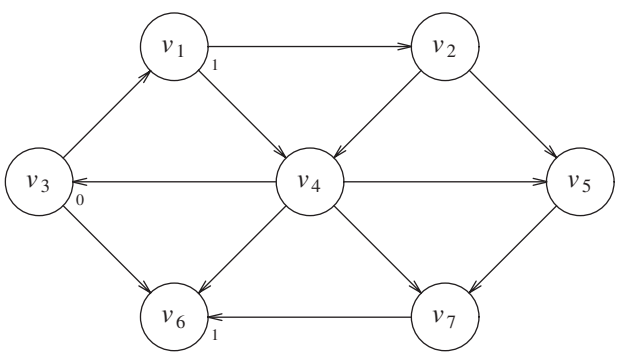

Figure 9.12 Graph after finding all vertices whose path length from s is 1

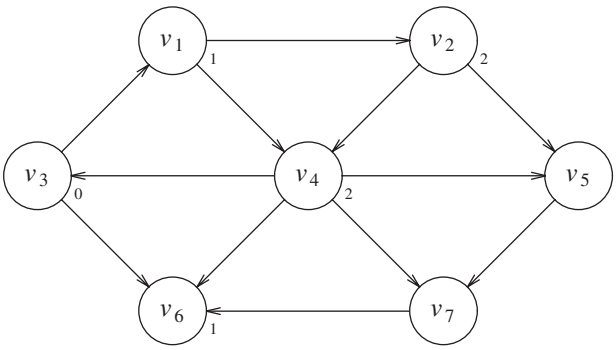

Figure 9.13 Graph after finding all vertices whose shortest path is 2

already known. This search tells us that the shortest path to v_2 and v_4 is 2. Figure 9.13 shows the progress that has been made so far.

Finally we can find, by examining vertices adjacent to the recently evaluated v_2 and v_4, that v_5 and v_7 have a shortest path of three edges. All vertices have now been calculated, and so Figure 9.14 shows the final result of the algorithm.

This strategy for searching a graph is known as **breadth-first search**. It operates by processing vertices in layers: The vertices closest to the start are evaluated first, and the most distant vertices are evaluated last. This is much the same as a level-order traversal for trees.

Given this strategy, we must translate it into code. Figure 9.15 shows the initial configuration of the table that our algorithm will use to keep track of its progress.

For each vertex, we will keep track of three pieces of information. First, we will keep its distance from s in the entry d_v. Initially all vertices are unreachable except for s, whose path length is 0. The entry in p_v is the bookkeeping variable, which will allow us to print the actual paths. The entry *known* is set to `true` after a vertex is processed. Initially, all entries are not *known,* including the start vertex. When a vertex is marked *known,* we have

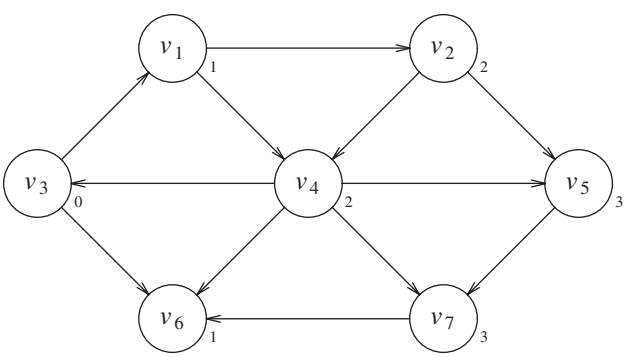

Figure 9.14 Final shortest paths

v	*known*	d_v	p_v
v_1	F	∞	0
v_2	F	∞	0
v_3	F	0	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0

Figure 9.15 Initial configuration of table used in unweighted shortest-path computation

a guarantee that no cheaper path will ever be found, and so processing for that vertex is essentially complete.

The basic algorithm can be described in Figure 9.16. The algorithm in Figure 9.16 mimics the diagrams by declaring as *known* the vertices at distance $d = 0$, then $d = 1$, then $d = 2$, and so on, and setting all the adjacent vertices w that still have $d_w = \infty$ to a distance $d_w = d + 1$.

```
void Graph::unweighted( Vertex s )
{
    for each Vertex v
    {
        v.dist = INFINITY;
        v.known = false;
    }

    s.dist = 0;

    for( int currDist = 0; currDist < NUM_VERTICES; currDist++ )
        for each Vertex v
            if( !v.known && v.dist == currDist )
            {
                v.known = true;
                for each Vertex w adjacent to v
                    if( w.dist == INFINITY )
                    {
                        w.dist = currDist + 1;
                        w.path = v;
                    }
            }
}
```

Figure 9.16 Pseudocode for unweighted shortest-path algorithm

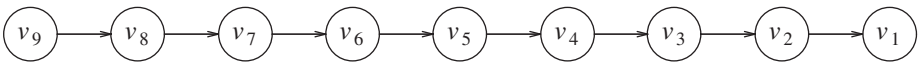

Figure 9.17 A bad case for unweighted shortest-path algorithm using Figure 9.16

By tracing back through the p_v variable, the actual path can be printed. We will see how when we discuss the weighted case.

The running time of the algorithm is $O(|V|^2)$, because of the doubly nested `for` loops. An obvious inefficiency is that the outside loop continues until `NUM_VERTICES-1`, even if all the vertices become *known* much earlier. Although an extra test could be made to avoid this, it does not affect the worst-case running time, as can be seen by generalizing what happens when the input is the graph in Figure 9.17 with start vertex v_9.

We can remove the inefficiency in much the same way as was done for topological sort. At any point in time, there are only two types of *unknown* vertices that have $d_v \neq \infty$. Some have $d_v =$ `currDist`, and the rest have $d_v =$ `currDist + 1`. Because of this extra structure, it is very wasteful to search through the entire table to find a proper vertex.

A very simple but abstract solution is to keep two boxes. Box #1 will have the unknown vertices with $d_v =$ `currDist`, and box #2 will have $d_v =$ `currDist + 1`. The test to find an appropriate vertex v can be replaced by finding any vertex in box #1. After updating w (inside the innermost `if` block), we can add w to box #2. After the outermost `for` loop terminates, box #1 is empty, and box #2 can be transferred to box #1 for the next pass of the `for` loop.

We can refine this idea even further by using just one queue. At the start of the pass, the queue contains only vertices of distance `currDist`. When we add adjacent vertices of distance `currDist + 1`, since they enqueue at the rear, we are guaranteed that they will not be processed until after all the vertices of distance `currDist` have been processed. After the last vertex at distance `currDist` dequeues and is processed, the queue only contains vertices of distance `currDist + 1`, so this process perpetuates. We merely need to begin the process by placing the start node on the queue by itself.

The refined algorithm is shown in Figure 9.18. In the pseudocode, we have assumed that the start vertex, s, is passed as a parameter. Also, it is possible that the queue might empty prematurely, if some vertices are unreachable from the start node. In this case, a distance of `INFINITY` will be reported for these nodes, which is perfectly reasonable. Finally, the `known` data member is not used; once a vertex is processed it can never enter the queue again, so the fact that it need not be reprocessed is implicitly marked. Thus, the `known` data member can be discarded. Figure 9.19 shows how the values on the graph we have been using are changed during the algorithm (it includes the changes that would occur to `known` if we had kept it).

Using the same analysis as was performed for topological sort, we see that the running time is $O(|E| + |V|)$, as long as adjacency lists are used.

9.3.2 Dijkstra's Algorithm

If the graph is weighted, the problem (apparently) becomes harder, but we can still use the ideas from the unweighted case.

```
void Graph::unweighted( Vertex s )
{
    Queue<Vertex> q;

    for each Vertex v
        v.dist = INFINITY;

    s.dist = 0;
    q.enqueue( s );

    while( !q.isEmpty( ) )
    {
        Vertex v = q.dequeue( );

        for each Vertex w adjacent to v
            if( w.dist == INFINITY )
            {
                w.dist = v.dist + 1;
                w.path = v;
                q.enqueue( w );
            }
    }
}
```

Figure 9.18 Psuedocode for unweighted shortest-path algorithm

We keep all of the same information as before. Thus, each vertex is marked as either *known* or *unknown*. A tentative distance d_v is kept for each vertex, as before. This distance turns out to be the shortest path length from s to v using only *known* vertices as intermediates. As before, we record p_v, which is the last vertex to cause a change to d_v.

The general method to solve the single-source shortest-path problem is known as **Dijkstra's algorithm.** This thirty-year-old solution is a prime example of a **greedy algorithm.** Greedy algorithms generally solve a problem in stages by doing what appears to be the best thing at each stage. For example, to make change in U.S. currency, most people count out the quarters first, then the dimes, nickels, and pennies. This greedy algorithm gives change using the minimum number of coins. The main problem with greedy algorithms is that they do not always work. The addition of a 12-cent piece breaks the coin-changing algorithm for returning 15 cents, because the answer it gives (one 12-cent piece and three pennies) is not optimal (one dime and one nickel).

Dijkstra's algorithm proceeds in stages, just like the unweighted shortest-path algorithm. At each stage, Dijkstra's algorithm selects a vertex, v, which has the smallest d_v among all the *unknown* vertices and declares that the shortest path from s to v is *known*. The remainder of a stage consists of updating the values of d_w.

In the unweighted case, we set $d_w = d_v + 1$ if $d_w = \infty$. Thus, we essentially lowered the value of d_w if vertex v offered a shorter path. If we apply the same logic to the weighted

	Initial State			v_3 Dequeued			v_1 Dequeued			v_6 Dequeued		
v	*known*	d_v	p_v	*known*	d_v	p_v	*known*	d_v	p_v	*known*	d_v	p_v
v_1	F	∞	0	F	1	v_3	T	1	v_3	T	1	v_3
v_2	F	∞	0	F	∞	0	F	2	v_1	F	2	v_1
v_3	F	0	0	T	0	0	T	0	0	T	0	0
v_4	F	∞	0	F	∞	0	F	2	v_1	F	2	v_1
v_5	F	∞	0	F	∞	0	F	∞	0	F	∞	0
v_6	F	∞	0	F	1	v_3	F	1	v_3	T	1	v_3
v_7	F	∞	0	F	∞	0	F	∞	0	F	∞	0
Q:	v_3			v_1, v_6			v_6, v_2, v_4			v_2, v_4		

	v_2 Dequeued			v_4 Dequeued			v_5 Dequeued			v_7 Dequeued		
v	*known*	d_v	p_v	*known*	d_v	p_v	*known*	d_v	p_v	*known*	d_v	p_v
v_1	T	1	v_3	T	1	v_3	T	1	v_3	T	1	v_3
v_2	T	2	v_1	T	2	v_1	T	2	v_1	T	2	v_1
v_3	T	0	0	T	0	0	T	0	0	T	0	0
v_4	F	2	v_1	T	2	v_1	T	2	v_1	T	2	v_1
v_5	F	3	v_2	F	3	v_2	T	3	v_2	T	3	v_2
v_6	T	1	v_3	T	1	v_3	T	1	v_3	T	1	v_3
v_7	F	∞	0	F	3	v_4	F	3	v_4	T	3	v_4
Q:	v_4, v_5			v_5, v_7			v_7			empty		

Figure 9.19 How the data change during the unweighted shortest-path algorithm

case, then we should set $d_w = d_v + c_{v,w}$ if this new value for d_w would be an improvement. Put simply, the algorithm decides whether or not it is a good idea to use v on the path to w. The original cost, d_w, is the cost without using v; the cost calculated above is the cheapest path using v (and only *known* vertices).

The graph in Figure 9.20 is our example. Figure 9.21 represents the initial configuration, assuming that the start node, s, is v_1. The first vertex selected is v_1, with path length 0. This vertex is marked *known*. Now that v_1 is *known*, some entries need to be adjusted. The vertices adjacent to v_1 are v_2 and v_4. Both these vertices get their entries adjusted, as indicated in Figure 9.22.

Next, v_4 is selected and marked *known*. Vertices v_3, v_5, v_6, and v_7 are adjacent, and it turns out that all require adjusting, as shown in Figure 9.23.

Next, v_2 is selected. v_4 is adjacent but already known, so no work is performed on it. v_5 is adjacent but not adjusted, because the cost of going through v_2 is $2 + 10 = 12$ and a path of length 3 is already known. Figure 9.24 shows the table after these vertices are selected.

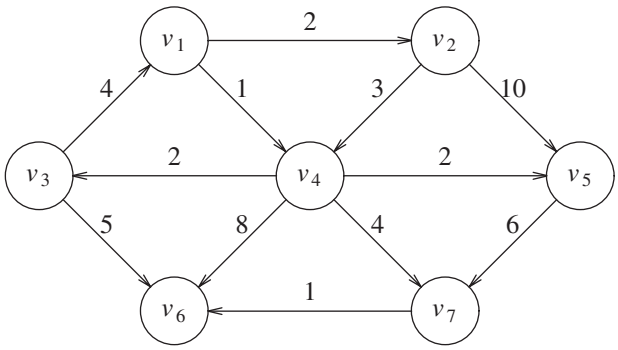

Figure 9.20 The directed graph G (again)

v	*known*	d_v	p_v
v_1	F	0	0
v_2	F	∞	0
v_3	F	∞	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0

Figure 9.21 Initial configuration of table used in Dijkstra's algorithm

v	*known*	d_v	p_v
v_1	T	0	0
v_2	F	2	v_1
v_3	F	∞	0
v_4	F	1	v_1
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0

Figure 9.22 After v_1 is declared *known*

The next vertex selected is v_5 at cost 3. v_7 is the only adjacent vertex, but it is not adjusted, because $3 + 6 > 5$. Then v_3 is selected, and the distance for v_6 is adjusted down to $3 + 5 = 8$. The resulting table is depicted in Figure 9.25.

Next, v_7 is selected; v_6 gets updated down to $5 + 1 = 6$. The resulting table is Figure 9.26.

v	*known*	d_v	p_v
v_1	T	0	0
v_2	F	2	v_1
v_3	F	3	v_4
v_4	T	1	v_1
v_5	F	3	v_4
v_6	F	9	v_4
v_7	F	5	v_4

Figure 9.23 After v_4 is declared *known*

v	*known*	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	F	3	v_4
v_4	T	1	v_1
v_5	F	3	v_4
v_6	F	9	v_4
v_7	F	5	v_4

Figure 9.24 After v_2 is declared *known*

v	*known*	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	3	v_4
v_4	T	1	v_1
v_5	T	3	v_4
v_6	F	8	v_3
v_7	F	5	v_4

Figure 9.25 After v_5 and then v_3 are declared *known*

Finally, v_6 is selected. The final table is shown in Figure 9.27. Figure 9.28 graphically shows how edges are marked *known* and vertices updated during Dijkstra's algorithm.

To print out the actual path from a start vertex to some vertex v, we can write a recursive routine to follow the trail left in the p variables.

We now give pseudocode to implement Dijkstra's algorithm. Each `Vertex` stores various data members that are used in the algorithm. This is shown in Figure 9.29.

v	*known*	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	3	v_4
v_4	T	1	v_1
v_5	T	3	v_4
v_6	F	6	v_7
v_7	T	5	v_4

Figure 9.26 After v_7 is declared *known*

v	*known*	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	3	v_4
v_4	T	1	v_1
v_5	T	3	v_4
v_6	T	6	v_7
v_7	T	5	v_4

Figure 9.27 After v_6 is declared *known* and algorithm terminates

The path can be printed out using the recursive routine in Figure 9.30. The routine recursively prints the path all the way up to the vertex before v on the path, and then just prints v. This works because the path is simple.

Figure 9.31 shows the main algorithm, which is just a `for` loop to fill up the table using the greedy selection rule.

A proof by contradiction will show that this algorithm always works as long as no edge has a negative cost. If any edge has negative cost, the algorithm could produce the wrong answer (see Exercise 9.7(a)). The running time depends on how the vertices are manipulated, which we have yet to consider. If we use the obvious algorithm of sequentially scanning the vertices to find the minimum d_v, each phase will take $O(|V|)$ time to find the minimum, and thus $O(|V|^2)$ time will be spent finding the minimum over the course of the algorithm. The time for updating d_w is constant per update, and there is at most one update per edge for a total of $O(|E|)$. Thus, the total running time is $O(|E| + |V|^2) = O(|V|^2)$. If the graph is dense, with $|E| = \Theta(|V|^2)$, this algorithm is not only simple but also essentially optimal, since it runs in time linear in the number of edges.

If the graph is sparse, with $|E| = \Theta(|V|)$, this algorithm is too slow. In this case, the distances would need to be kept in a priority queue. There are actually two ways to do this; both are similar.

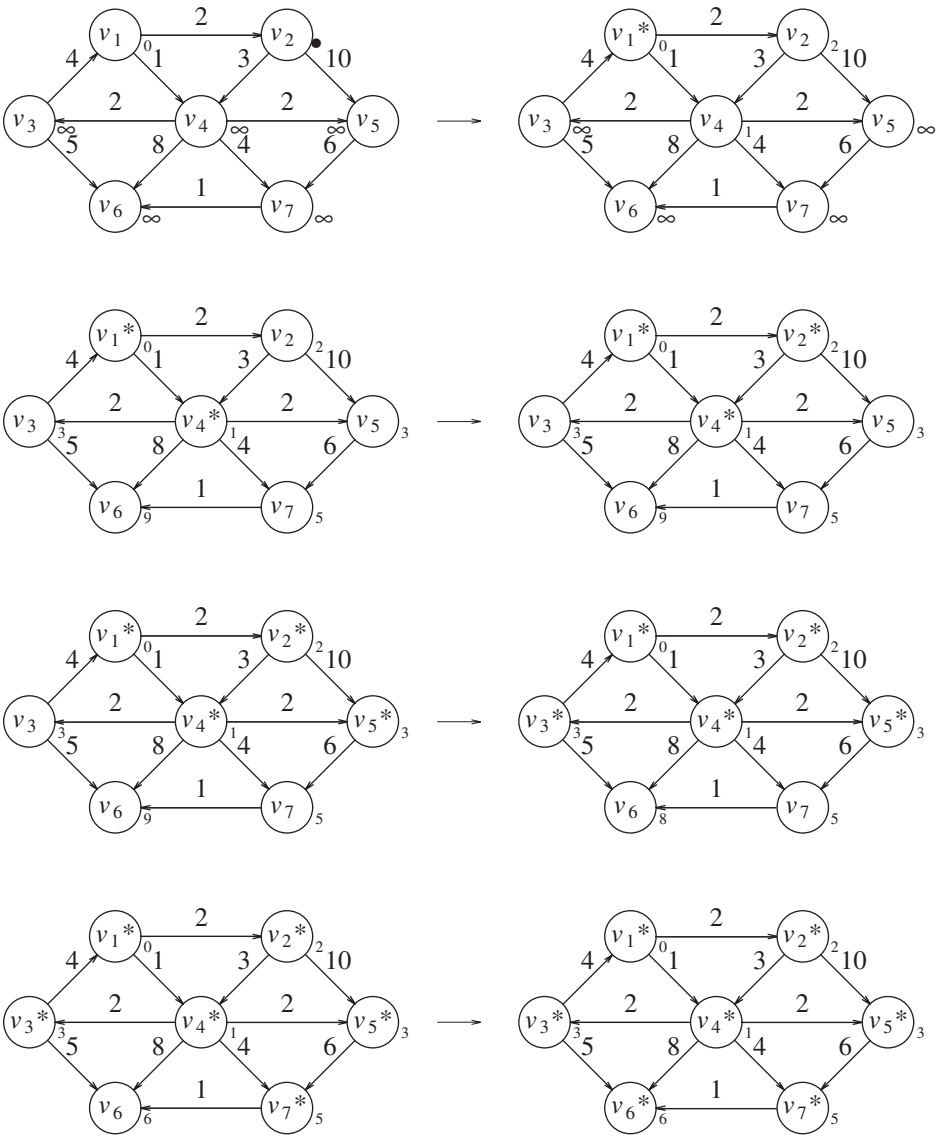

Figure 9.28 Stages of Dijkstra's algorithm

Selection of the vertex v is a `deleteMin` operation, since once the unknown minimum vertex is found, it is no longer unknown and must be removed from future consideration. The update of w's distance can be implemented two ways.

One way treats the update as a `decreaseKey` operation. The time to find the minimum is then $O(\log |V|)$, as is the time to perform updates, which amount to `decreaseKey` operations. This gives a running time of $O(|E| \log |V| + |V| \log |V|) = O(|E| \log |V|)$, an improvement

```
/**
 * PSEUDOCODE sketch of the Vertex structure.
 * In real C++, path would be of type Vertex *,
 * and many of the code fragments that we describe
 * require either a dereferencing * or use the
 * -> operator instead of the . operator.
 * Needless to say, this obscures the basic algorithmic ideas.
 */
struct Vertex
{
    List      adj;     // Adjacency list
    bool      known;
    DistType  dist;    // DistType is probably int
    Vertex    path;    // Probably Vertex *, as mentioned above
        // Other data and member functions as needed
};
```

Figure 9.29 `Vertex` class for Dijkstra's algorithm (pseudocode)

```
/**
 * Print shortest path to v after dijkstra has run.
 * Assume that the path exists.
 */
void Graph::printPath( Vertex v )
{
    if( v.path != NOT_A_VERTEX )
    {
        printPath( v.path );
        cout << " to ";
    }
    cout << v;
}
```

Figure 9.30 Routine to print the actual shortest path

over the previous bound for sparse graphs. Since priority queues do not efficiently support the `find` operation, the location in the priority queue of each value of d_i will need to be maintained and updated whenever d_i changes in the priority queue. If the priority queue is implemented by a binary heap, this will be messy. If a pairing heap (Chapter 12) is used, the code is not too bad.

An alternate method is to insert w and the new value d_w into the priority queue every time w's distance changes. Thus, there may be more than one representative for each vertex in the priority queue. When the `deleteMin` operation removes the smallest vertex from the priority queue, it must be checked to make sure that it is not already *known* and, if

```
void Graph::dijkstra( Vertex s )
{
    for each Vertex v
    {
        v.dist = INFINITY;
        v.known = false;
    }

    s.dist = 0;

    while( there is an unknown distance vertex )
    {
        Vertex v = smallest unknown distance vertex;

        v.known = true;

        for each Vertex w adjacent to v
            if( !w.known )
            {
                DistType cvw = cost of edge from v to w;

                if( v.dist + cvw < w.dist )
                {
                    // Update w
                    decrease( w.dist to v.dist + cvw );
                    w.path = v;
                }
            }
    }
}
```

Figure 9.31 Pseudocode for Dijkstra's algorithm

it is, it is simply ignored and another `deleteMin` is performed. Although this method is superior from a software point of view, and is certainly much easier to code, the size of the priority queue could get to be as large as $|E|$. This does not affect the asymptotic time bounds, since $|E| \leq |V|^2$ implies that $\log |E| \leq 2 \log |V|$. Thus, we still get an $O(|E| \log |V|)$ algorithm. However, the space requirement does increase, and this could be important in some applications. Moreover, because this method requires $|E|$ `deleteMin`s instead of only $|V|$, it is likely to be slower in practice.

Notice that for the typical problems, such as computer mail and mass transit commutes, the graphs are typically very sparse because most vertices have only a couple of edges, so it is important in many applications to use a priority queue to solve this problem.

There are better time bounds possible using Dijkstra's algorithm if different data structures are used. In Chapter 11, we will see another priority queue data structure called the